Name: Student ID:

# Question 1

Use the `gapminder` data frame in the `gapminder` library to recreate a version of Hans Rosling's famous data visualization shown in the *A Grammar of Graphics* slides (a single plot instead of a movie; in other words, for just a single year).

- First, run `?gapminder` to see the help page for the dataset, which explains the columns and their meaning. You don't need to include this in your final rendered file, but take a look before you get started.
- You can see at a glance which years are available by running `count(gapminder, year)` in an R chunk.
  - Side note: `count` is a useful function, from tidyverse, that takes a dataset and a column name as arguments. It produces a table saying how many rows had each possible value of that column. In this case, it will tell you how many observations (countries) there are for each year in the `gapminder` dataset.
- **Trim the data frame down to only a year's worth of observations.** You may use the the command below (we discuss `filter` in the next lesson). In this instance, we've picked the year 1987, but you can use any year in the data frame that you'd like! (Note that not all years have data, so pick one that exists).

```
gapminder_one_year <- filter(gapminder, year == 1987)
```

- In place of fertility rate use GDP per capita.

Constructing this plot requires several distinct steps.

1. Determine the correct geometry for the plot and make an initial `ggplot` with the two variables on the x and y axis.
2. Distinguish the points from different continents by either `shape` or `color`. Which ever one you do not use, *set* its value to something other than the default.

- Color: If you map the `color` aesthetic to a column, each unique value in that column will be represented by a different color.
  - If you want to make all (say) points use the same color, you *set* `color` the color you want. `ggplot` supports most color options like "red", "blue", and "green" (put them in quotes). If you're familiar with HTML (hex codes)[https://htmlcolorcodes.com/], you can use those as well. as well as.
  - You can also use the `scale_color_brewer()` function to choose from a palette of colors. See here for more information.
- Shape: by default, `geom_point` draws dots (filled circles) for each point. But you can change this to triangles, or squares, or something else with the `shape` aesthetic. If you *map* the `shape` aesthetic to a column, all points with the same value of that column will have the same shape. Try it out and see.
  - To make all points have one specific shape, you *set* `shape` to a *number* between 0 and 25. See here for all of your options. For example, `shape = 2` will give you empty triangles, and `shape = 15` will give you filled squares.

3. Alter the x and y axis labels so that they're more descriptive than just the variable given names in `gapminder`.
4. Add an annotation that draws attention to a particular feature of the data (of your choosing).
5. Title your plot with a claim based on your data.

6. Apply a theme of your choosing.

Use RStudio to write the code and see your visualization. Once you are happy with it, handwrite your code in the space below.

## Question 2

Revisit the plot you made for Lab 2 (Class Survey), Question 8. This time, incorporate at least 2 of the elements from the *Communicating with Graphics* section of the *Grammar of Graphics* tutorial to polish your final plot there into one that tells a more focused story. No need to copy that code here, but in the space below, write a list of the changes that you applied to the plot.